

# Strukturalni paterni

Za naš projekat koristit ćemo Facade i Decorator pattern-e.

## Facade Pattern

### Opis:

Prikazivanje detalja o seriji ili filmu u sistemu zahtijeva podatke iz više povezanih klasa: Sezona, Epizoda, Uloga, Glumac i Recenzija. Bez fasade, klijent bi morao direktno upravljati svim ovim objektima, što stvara preveliku kompleksnost i zavisnost.

### Primjena u sistemu:

Uvodi se klasa **KatalogFacade** koja klijentu nudi jednu metodu (npr. **dohvatiSveDetalje(id)**). Ona interno upravlja radom svih navedenih klasa i klijentu vraća spreman set podataka, sakrivajući složenost unutrašnjih veza sistema.

## Decorator Pattern

### Opis:

Dinamičko dodavanje osobina oglasima (poput vizuelnih stilova ili prioriternog rangiranja) bez kreiranja velikog broja podklasa za svaku moguću kombinaciju osobina.

### Primjena u sistemu:

Uvodi se **OglasDecorator** hijerarhija. Osnovna klasa Oglas ostaje fokusirana na podatke, dok konkretni dekoratori (npr. **PremiumOglasDecorator**) proširuju ponašanje metode **prikaziOglas()** dodajući joj specifične vizuelne ili logičke attribute u zavisnosti od potreba kampanje.

Preostali paterni koje nećemo koristiti:

## **Adapter Pattern**

Adapter pattern služi da se postojeći objekat prilagodi za korištenje na neki drugi način, bez mijenjanja same definicije objekta. Na taj način obezbjeđuje se da će se objekti i dalje moći upotrebljavati na način kako su se dosad upotrebljavali, a u isto vrijeme će se omogućiti njihovo prilagođavanje novim uslovima.

## **Bridge Pattern**

Bridge pattern služi kako bi se apstrakcija nekog objekta odvojila od njegove implementacije. Ovaj pattern veoma je važan jer omogućava ispunjavanje Open-Closed SOLID principa, odnosno uz poštivanje ovog patterna omogućava se nadogradnja klasnog modela u budućnosti te osigurava da se neće morati vršiti određene promjene u postojećim klasama.

## **Composite pattern**

Composite pattern služi za kreiranje hijerarhije objekata. Koristi se kada svi objekti imaju različite implementacije nekih metoda, no potrebno im je svima pristupati na isti način, te se na taj način pojednostavljuje njihova implementacija.

## **Proxy pattern**

Proxy pattern služi za dodatno osiguravanje objekata od pogrešne ili zlonamjerne upotrebe. Primjenom ovog patterna omogućava se kontrola pristupa objektima, te se onemogućava manipulacija objektima ukoliko neki uslov nije ispunjen, odnosno ukoliko korisnik nema prava pristupa traženom objektu.